

Σεμινάριο: Πώς λειτουργεί το νευρωνικό δίκτυο LSTM

Από τον έναν νευρώνα μέχρι το μοντέλο συναλλαγών μας — για αρχάριους

Dr. Stefanos Drakos — QUANTIQ / AGEL AI

Ιούνιος 2026

Αυτό το φυλλάδιο εξηγεί από το μηδέν πώς δουλεύει ένα νευρωνικό δίκτυο, γιατί χρειαζόμαστε τον ειδικό τύπο που λέγεται **LSTM** όταν έχουμε χρονοσειρές, και — το σημαντικότερο — πώς ακριβώς το χρησιμοποιούμε εμείς για να παίρνει αποφάσεις συναλλαγών. Δεν προϋποθέτει τίποτα πέρα από βασική άλγεβρα και την έννοια του μέσου όρου. Όπου υπάρχουν εξισώσεις, τις συνοδεύει πάντα ένα μικρό αριθμητικό παράδειγμα.

Τι θα δούμε

Μέρος Α – η θεωρία: ο νευρώνας, τα στρώματα, και πώς ένα δίκτυο «μαθαίνει».

Μέρος Β – γιατί LSTM: η μνήμη στις χρονοσειρές και οι τρεις «πύλες».

Μέρος Γ – η δική μας περίπτωση: το Deep Momentum Network, η «απώλεια Sharpe», τίμια εκπαίδευση.

1. Ο νευρώνας: το θεμέλιο

Ένα νευρωνικό δίκτυο φτιάχνεται από πολλές μικρές, πανομοιότυπες μονάδες που λέγονται **νευρώνες**. Ένας νευρώνας κάνει κάτι απλό: παίρνει κάποιους αριθμούς-εισόδους, δίνει σε καθέναν μια **βαρύτητα** (πόσο σημαντικός είναι), τους αθροίζει, προσθέτει μια σταθερά, και περνάει το αποτέλεσμα από μια συνάρτηση. Σκεφτείτε τον σαν έναν κριτή που ζυγίζει στοιχεία: κάποια μετράνε θετικά, κάποια αρνητικά, και στο τέλος βγάζει μια ετυμηγορία.

Μαθηματικά, αν οι είσοδοι είναι $x_1, \dots, x_n$ με βαρύτητες $w_1, \dots, w_n$ και σταθερά (bias) b :

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b, \quad y = \varphi(z).$$

Το z είναι το «ζυγισμένο άθροισμα» και το φ είναι η **συνάρτηση ενεργοποίησης**.

Παράδειγμα. Έστω δύο είσοδοι: μια ένδειξη τάσης $x_1 = 0,5$ και μια ένδειξη μεταβλητότητας $x_2 = 0,2$. Ο νευρώνας έχει μάθει βαρύτητες $w_1 = 0,8$ (η τάση μετράει θετικά) και $w_2 = -1,0$ (η μεγάλη μεταβλητότητα μετράει αρνητικά), με $b = 0,1$. Τότε

$z = 0,8 \cdot 0,5 - 1,0 \cdot 0,2 + 0,1 = 0,3$. Αν $\varphi = \tanh$, η έξοδος είναι $\tanh(0,3) \approx 0,29$ — μια ήπια «θετική» τοποθέτηση.

Γιατί χρειαζόμαστε τη συνάρτηση ενεργοποίησης

Χωρίς το φ , ο νευρώνας θα ήταν απλώς μια ευθεία γραμμή, και όσους κι αν στοιβάσαμε, το σύνολο θα παρέμενε μια ευθεία. Η φ βάζει την **καμπύλη** που επιτρέπει στο δίκτυο να μαθαίνει περίπλοκα μοτίβα. Τρεις συνηθισμένες επιλογές:

- **Sigmoid** $\sigma(z) = \frac{1}{1 + e^{-z}}$ — στριμώνει κάθε αριθμό στο διάστημα $(0, 1)$. Ιδανική για «πόσο ανοιχτή είναι μια πύλη» (0 = κλειστή, 1 = ανοιχτή).
- **tanh** — στριμώνει στο $(-1, 1)$. Ιδανική για «θέση»: -1 πλήρης πώληση, $+1$ πλήρης αγορά.
- **ReLU** $\max(0, z)$ — αφήνει τα θετικά, μηδενίζει τα αρνητικά· φθηνή και δημοφιλής στα βαθιά δίκτυα.

2. Στρώματα και πώς το δίκτυο «μαθαίνει»

Έναν νευρώνα μόνο του δεν τον λέμε δίκτυο. Βάζουμε πολλούς σε μια σειρά (ένα **στρώμα**), και στοιβάζουμε στρώματα το ένα πάνω στο άλλο: η έξοδος του ενός γίνεται είσοδος του επόμενου. Το πέρασμα των αριθμών από την είσοδο ως την τελική έξοδο λέγεται **forward pass**. Στην αρχή οι βαρύτητες είναι τυχαίες, άρα το δίκτυο βγάζει ανοησίες. Η «μάθηση» είναι η διαδικασία που τις διορθώνει.

Η συνάρτηση κόστους

Χρειαζόμαστε έναν αριθμό που να λέει «πόσο λάθος είσαι» — τη **συνάρτηση κόστους** (loss). Όσο μικρότερη, τόσο καλύτερο το δίκτυο. Η εκπαίδευση είναι, κυριολεκτικά, η αναζήτηση των βαρυτήτων που ελαχιστοποιούν αυτό το κόστος.

Κατάβαση κλίσης (gradient descent)

Φανταστείτε το κόστος σαν ένα τοπίο με λόφους και κοιλάδες, όπου κάθε σημείο είναι μια επιλογή βαρυτήτων. Θέλουμε τον πάτο της κοιλάδας. Σε κάθε βήμα υπολογίζουμε προς τα πού «κατηφορίζει» το έδαφος (την **κλίση**) και κάνουμε ένα μικρό βήμα προς τα κάτω:

$$w \leftarrow w - \eta \frac{\partial \text{Loss}}{\partial w},$$

όπου η είναι ο **ρυθμός μάθησης** (πόσο μεγάλο βήμα). Επαναλαμβάνουμε χιλιάδες φορές.

Κλειδί: Ο αλγόριθμος που υπολογίζει αποδοτικά αυτές τις κλίσεις, ξετυλίγοντας τον κανόνα της αλυσίδας από την έξοδο προς τα πίσω, λέγεται **backpropagation**. Είναι ο λόγος που τα νευρωνικά δίκτυα είναι πρακτικά: δεν δοκιμάζουμε βαρύτητες στην τύχη, ξέρουμε προς τα πού να τις σπρώξουμε.

3. Γιατί LSTM: το πρόβλημα της μνήμης

Όλα τα παραπάνω δουλεύουν όταν κάθε παράδειγμα είναι ανεξάρτητο. Όμως οι τιμές μιας αγοράς είναι **ακολουθία**: η σημερινή τιμή έχει νόημα μόνο σε σχέση με τις προηγούμενες. Ένα απλό δίκτυο δεν έχει μνήμη — βλέπει κάθε μέρα σαν να ήταν η πρώτη. Χρειαζόμαστε κάτι που να **θυμάται**.

Τα επαναλαμβανόμενα δίκτυα (RNN)

Η πρώτη ιδέα: να δώσουμε στο δίκτυο μια «κρυφή κατάσταση» h_t που μεταφέρεται από τη μια χρονική στιγμή στην επόμενη. Σε κάθε βήμα, η νέα κατάσταση εξαρτάται από τη σημερινή είσοδο x_t και από τη χθεσινή κατάσταση h_{t-1} . Έτσι το δίκτυο κουβαλάει πληροφορία από το παρελθόν. Το πρόβλημα: σε μεγάλες ακολουθίες η πληροφορία «σβήνει» καθώς περνάει από πολλά βήματα (το λεγόμενο *vanishing gradient*) — το δίκτυο ξεχνάει ό,τι συνέβη πριν από πολύ καιρό.

Το κελί LSTM: μια ελεγχόμενη μνήμη

Το LSTM (*Long Short-Term Memory*) λύνει ακριβώς αυτό. Προσθέτει μια ξεχωριστή «ταινία μνήμης», την **κατάσταση κελιού** C_t , που διατρέχει όλη την ακολουθία με ελάχιστες αλλαγές — σαν ένα σημειωματάριο που κρατάει σημειώσεις για πολλή ώρα. Τρεις **πύλες** αποφασίζουν τι γράφεται, τι σβήνεται και τι διαβάζεται. Κάθε πύλη είναι ένας μικρός νευρώνας με sigmoid, άρα βγάζει έναν αριθμό στο $(0, 1)$: 0 σημαίνει «κλειστή», 1 σημαίνει «τελείως ανοιχτή».

Συμβολίζουμε με $[h_{t-1}, x_t]$ τη συνένωση χθεσινής κατάστασης και σημερινής εισόδου, και με $\odot$ τον πολλαπλασιασμό στοιχείο-προς-στοιχείο. Οι εξισώσεις του κελιού:

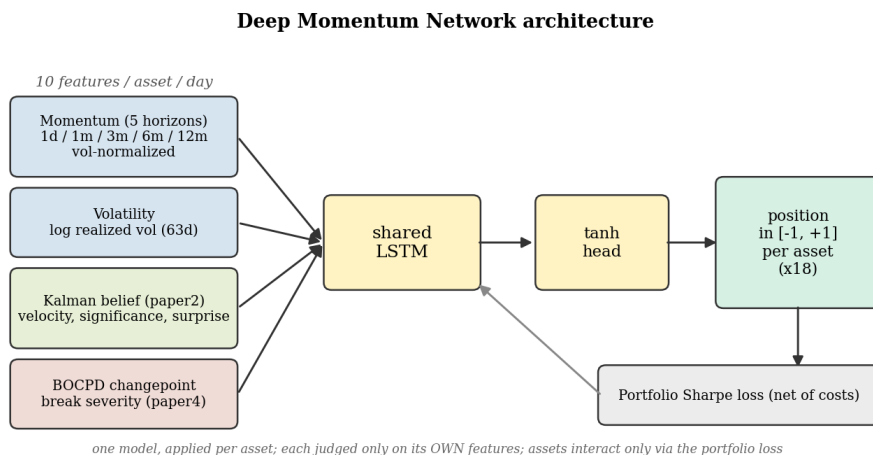
$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$	πύλη λήθης : τι κρατάμε από τη μνήμη
$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$	πύλη εισόδου : πόσο γράφουμε από το νέο
$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$	υποψήφια νέα πληροφορία
$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$	ενημέρωση της μνήμης
$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$	πύλη εξόδου : τι διαβάζουμε τώρα
$h_t = o_t \odot \tanh(C_t)$	η έξοδος αυτού του βήματος

Με απλά λόγια: η πύλη λήθης f_t σβήνει ό,τι δεν χρειάζεται πια· η πύλη εισόδου i_t προσθέτει τη χρήσιμη νέα πληροφορία· και η πύλη εξόδου o_t διαλέγει ποιο κομμάτι της μνήμης είναι σχετικό για την απόφαση *τώρα*. Το δίκτυο **μαθαίνει μόνο του** πότε να ανοίγει την κάθε πύλη.

Παράδειγμα. Έστω ότι η αγορά βρισκόταν σε ισχυρή ανοδική τάση και η μνήμη το έχει καταγράψει. Σήμερα η πύλη λήθης υπολογίζει $z = 2,0$, οπότε $f_t = \sigma(2,0) \approx 0,88$: κρατάμε το 88% της παλιάς μνήμης — η τάση παραμένει. Αν αντί γι' αυτό ο ανιχνευτής αλλαγής έδειχνε ότι κάτι έσπασε, το δίκτυο θα είχε μάθει να βγάζει αρνητικό z , οπότε f_t κοντά στο 0: «ξέχνα την παλιά τάση, ξεκίνα από την αρχή».

4. Η δική μας περίπτωση: το Deep Momentum Network

Τώρα που ξέρουμε τι είναι ένα LSTM, ας δούμε πώς ακριβώς το χρησιμοποιούμε. Το μοντέλο μας λέγεται **Deep Momentum Network** και η δομή του είναι λιτή:



Σχήμα 1: Για κάθε προϊόν, δέκα πληροφορίες ανά ημέρα τροφοδοτούν ένα LSTM· η έξοδος περνάει από μια tanh και γίνεται θέση από -1 (πλήρης πώληση) έως $+1$ (πλήρης αγορά).

Είσοδος, δίκτυο, έξοδος

Για κάθε προϊόν ξεχωριστά, η είσοδος είναι μια ακολουθία από **δέκα χαρακτηριστικά** ανά ημέρα: πέντε αποδόσεις διαφορετικού ορίζοντα κανονικοποιημένες με τη μεταβλητότητα, η ίδια η μεταβλητότητα, τρεις δείκτες «πεποίθησης» από ένα φίλτρο Kalman, και ένα σήμα αλλαγής καθεστώτος (BOCPD). Αυτή η ακολουθία περνάει από ένα **κοινό** LSTM, μετά από ένα γραμμικό στρώμα, και τέλος από μια tanh που δίνει την τελική θέση στο $[-1, 1]$:

$$\text{θέση}_t = \tanh(\text{Linear}(h_t)) \in [-1, 1].$$

«Κοινό» σημαίνει ότι το ίδιο δίκτυο εφαρμόζεται σε όλα τα προϊόντα — μαθαίνει έναν γενικό κανόνα «πώς μοιάζει μια αξιοποιήσιμη τάση», όχι ξεχωριστό κανόνα ανά μετοχή.

Κλειδί: Προσέξτε τι δεν κάνει: δεν προσπαθεί να προβλέψει την αυριανή τιμή. Βγάζει κατευθείαν μια θέση. Δεν μας νοιάζει η ακριβής πρόβλεψη, μας νοιάζει η σωστή τοποθέτηση.

Η «απώλεια Sharpe»: ο έξυπνος στόχος

Εδώ κρύβεται το πιο ενδιαφέρον. Αντί να εκπαιδεύσουμε το δίκτυο να μαντεύει τιμές, το εκπαιδεύουμε να μεγιστοποιεί απευθείας την **ποιότητα του κέρδους** ολόκληρου του χαρτοφυλακίου. Αν θέσεις είναι οι τοποθετήσεις και απόδοι οι επόμενες αποδόσεις, η ημερήσια απόδοση του χαρτοφυλακίου (αφαιρώντας ένα κόστος για τις

αλλαγές θέσης, τον τζίρο) είναι ένα διάνυσμα port, και η συνάρτηση κόστους είναι το **αρνητικό** του Sharpe:

$$\text{Loss} = - \frac{\text{μέσος(port)}}{\text{τυπ. απόκλιση(port)}}.$$

Ελαχιστοποιώντας αυτό, μεγιστοποιούμε τον λόγο «κέρδος προς ρίσκο». Επειδή το port είναι η απόδοση όλου του καλαθιού μαζί, η **διαφοροποίηση μπαίνει μέσα στον ίδιο τον στόχο**: το δίκτυο ανταμείβεται όταν οι θέσεις του σε διαφορετικά προϊόντα αλληλοσυμπληρώνονται και τιμωρείται όταν στοιβάζουν το ίδιο ρίσκο. Το κόστος τζίρου το αποτρέπει από νευρικές, ακριβές αλλαγές.

Τίμια εκπαίδευση: χωρίς «κλεψιά»

Ένα μοντέλο που έχει δει το μέλλον μοιάζει ιδιοφυές — και είναι άχρηστο. Γι' αυτό εκπαιδεύουμε με **φωλιασμένη διαδοχική επικύρωση** (nested walk-forward): το δίκτυο μαθαίνει αποκλειστικά από το παρελθόν, διαλέγει τις ρυθμίσεις του (μέγεθος, κανονικοποίηση) σε ένα ενδιάμεσο κομμάτι επικύρωσης που δεν είναι το τεστ, και αξιολογείται μόνο σε μελλοντικά δεδομένα που δεν έχει αγγίξει. Προσθέτουμε και τεχνικές σταθερότητας, όπως το «ψαλίδισμα» των κλίσεων ώστε η εκπαίδευση να μην εκτροχιάζεται.

Προσοχή: Χωρίς αυτή την πειθαρχία, οποιοδήποτε δίκτυο δείχνει εντυπωσιακά αποτελέσματα στο χαρτί που *εξαφανίζονται* στην πραγματικότητα. Η τιμιότητα της αξιολόγησης είναι πιο σημαντική από την πολυπλοκότητα του μοντέλου.

Πώς δένει με τα υπόλοιπα

Τα δέκα χαρακτηριστικά δεν είναι ακατέργαστες τιμές: είναι ήδη «καταστάσεις πεποίθησης» από το φίλτρο Kalman και τον ανιχνευτή αλλαγής BOCPD. Έτσι το LSTM δεν ξεκινάει από το μηδέν — παίρνει ήδη επεξεργασμένες ενδείξεις τάσης και ρίσκου, και μαθαίνει πώς να τις μετατρέπει σε θέση. Στην πράξη αυτό το ML μοντέλο ξεπερνά τον σταθερό κανόνα σε ποιότητα κέρδους, με μετρημένο αλλά υπαρκτό πλεονέκτημα.

Τι να κρατήσετε

- Ένας **νευρώνας** ζυγίζει εισόδους και περνάει το άθροισμα από μια καμπύλη· ένα **δίκτυο** είναι πολλοί νευρώνες σε στρώματα.
- Η **μάθηση** είναι κατάβαση σε ένα τοπίο κόστους, με τις κλίσεις να υπολογίζονται από το backpropagation.
- Το **LSTM** προσθέτει ελεγχόμενη μνήμη με τρεις πύλες, ώστε να θυμάται τάσεις και να τις ξεχνάει όταν αλλάζει το καθεστώς.
- Στη **δική μας** περίπτωση, ένα κοινό LSTM μετατρέπει δέκα belief-state χαρακτηριστικά σε θέση $[-1, 1]$, εκπαιδευμένο να μεγιστοποιεί τον Sharpe όλου του χαρτοφυλακίου, με τίμια διαδοχική επικύρωση.

QUANTIQ / AGEL AI. Εκπαιδευτικό συνοδευτικό της εργασίας «*From Dead Cross-Sectional Momentum to Belief-State Deep Time-Series Momentum*» (Drakos 2026).